LINUX / SYSTEMS

systemd Visual Guide

Units, dependencies, targets, timers, and journald logging.

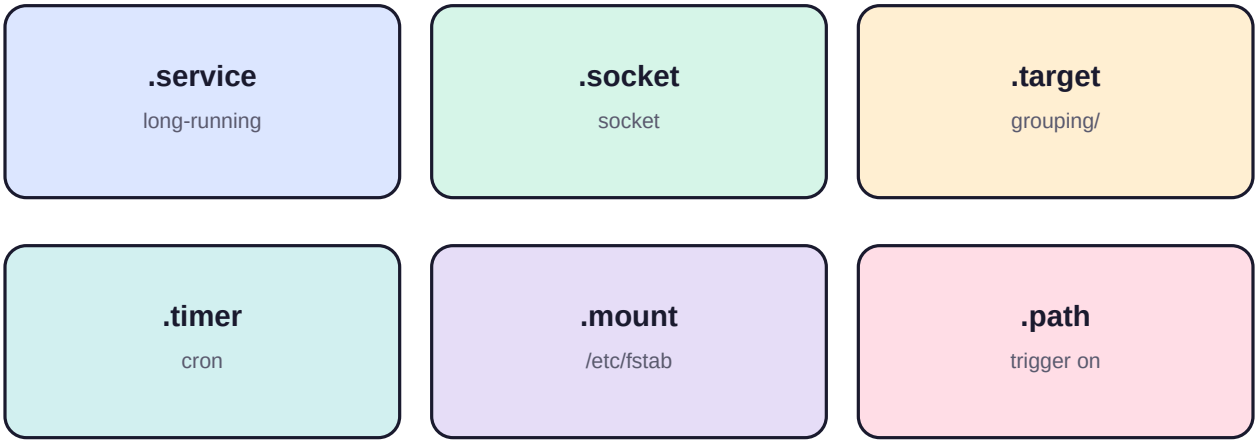
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Units: Everything Is a Unit

Services are just one unit type among many.



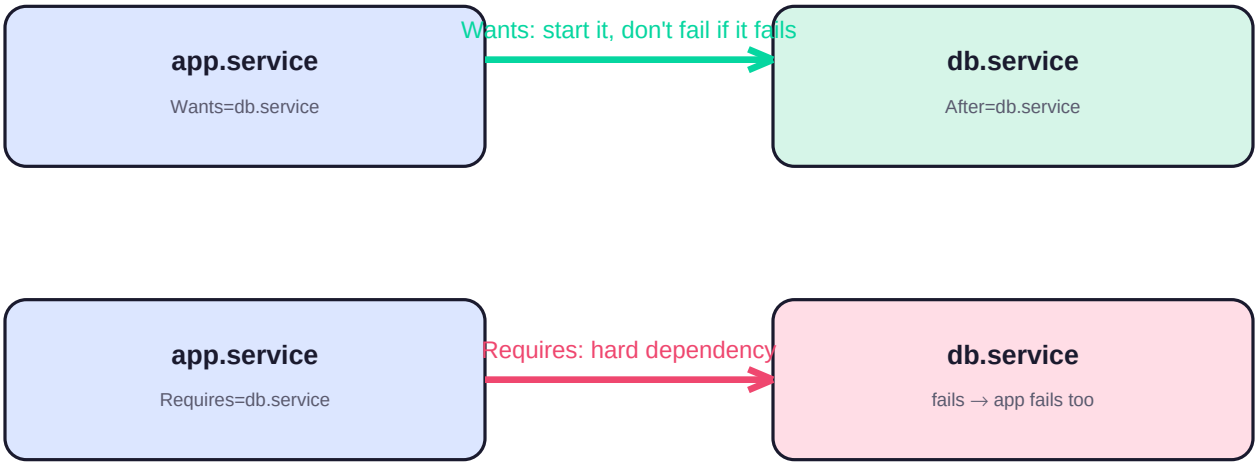

```
[Unit]
Description=My API
After=network-online.target

[Service]
ExecStart=/opt/api/server
Restart=always

[Install]
WantedBy=multi-user.target
```

A unit file = metadata (Unit) + behavior (Service) + enablement (Install)

STEP 2/6

Dependencies: Wants vs Requires

Ordering (After) is separate from requirement (Wants/Requires).



After= only orders startup — it does NOT pull the other unit in. You need Wants/Requires for that. Forgetting this is the #1 broken-unit cause.

```
$ systemctl list-dependencies app.service   # see the graph
```

STEP 3/6

Targets: Boot Milestones

Targets replaced SysV runlevels — they're just grouping units.

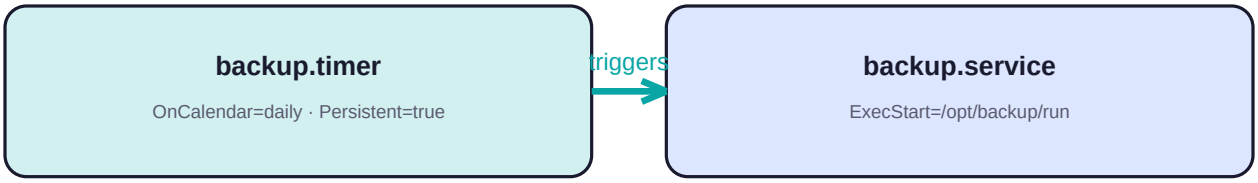
SysV runlevel	systemd target	Use
0	poweroff.target	halt
1	rescue.target	single user
3	multi-user.target	server
5	graphical.target	desktop
6	reboot.target	reboot

```
$ systemctl get-default          # boot target
$ systemctl isolate rescue.target   # drop to single-user live
```

STEP 4/6

Timers: cron, but With Dependencies

A .timer unit triggers a matching .service unit.



Timer wins over cron when…	cron wins when…
job must run AFTER network/db is up	one-liner on any unix
you need jitter, accuracy, calendars	no root/systemd (containers)
missed runs must catch up (Persistent)	you want zero unit files
logs must land in the journal	simplicity beats features

```
$ systemctl list-timers --all   # what runs when
```

STEP 5/6

journald: One Log to Rule Them All

Structured logging — query by unit, boot, priority, time.

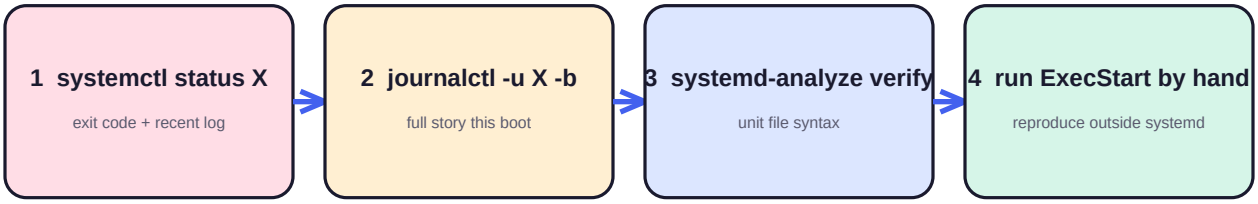
```
$ journalctl -u nginx.service -f        # follow one service
$ journalctl -p err -b                  # errors since this boot
$ journalctl --since '1 hour ago' -u app
$ journalctl -o json-pretty -u app | jq .MESSAGE
$ journalctl --disk-usage               # cap with SystemMaxUse=
```

- stdout/stderr of every service is captured automatically — no log files to rotate by hand.
- Logs are indexed by _SYSTEMD_UNIT, _PID, PRIORITY: filtering beats grep on text files.
- Volatile by default (/run/log/journal); set Storage=persistent to keep across reboots.
- Forward to syslog or a shipper (ForwardToSyslog) when you need central aggregation.

STEP 6/6

Debugging a Failed Unit

A repeatable 4-step drill.



After editing any unit: systemctl daemon-reload — systemd caches unit files and will happily run the stale copy.

```
$ systemctl is-failed '*.service'   # failed units right now
$ systemctl --failed
$ systemd-analyze blame | head     # what slowed this boot
```

STEP ✓/✓

Key Takeaways

systemd

- Units come in types (.service/.socket/.target/.timer/.mount/.path) — learn the family, not just services.
- After orders; Wants/Requires pull in. Ordering without requirement is the classic broken unit.
- Targets are boot milestones replacing runlevels; isolate rescue.target to debug.
- Timers beat cron when jobs need dependencies, catch-up, or journal logging.
- journalctl queries structured logs by unit/boot/priority — status, journal, verify, reproduce.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev